FORMATION

Analyse de données avec Python

Jour 3, après-midi
Gestion des dépendances et projet final

Cours, TP6 fil rouge et restitution devant le groupe

Le déroulé de l'après-midi

1. **Limites de pip**
2. **uv en détail**
3. **pyproject.toml** — puis TP6, étape 1
4. **Reproductibilité**
5. **Finitions**
6. **Pause**
7. **TP6** — étapes 2 à 7
8. **Restitution**

13h30
début

15h00
pause

17h00
fin

Le projet du TP6 est créé dès la section 3, pour que chacun parte avec un environnement prêt.

01

Les limites de pip et venv

Plusieurs outils | requirements.txt | Verrouillage

1 Ce qui coince quand le projet grossit

1

Plusieurs outils

`venv`, `pip`, `pip freeze`, et un autre outil pour changer de version de Python.

2

Une activation à ne pas oublier

Un `pip install` sans environnement activé installe au mauvais endroit.

3

Un requirements.txt ambigu

Dépendances directes et indirectes mélangées, développement et production confondus.

4

Un verrouillage tardif

`pip freeze` n'est qu'un instantané. Le format standard `pylock.toml` et `pip lock` sont récents.

Sur un projet avec beaucoup de dépendances, la résolution des versions compatibles par pip peut aussi être longue.

02

uv en détail

Commandes | add, sync, run | Version de Python

2 Les commandes courantes

Commande	Rôle	Commande	Rôle
`uv init --package`	créer un projet	`uv lock`	recalculer le verrou
`uv add pandas`	ajouter une dépendance	`uv tree`	arbre des dépendances
`uv add --dev pytest`	outil de développement	`uv python install 3.13`	installer Python
`uv remove pandas`	retirer	`uv python pin 3.13`	fixer la version
`uv sync`	installer depuis le verrou	`uvx ruff check .`	outil ponctuel
`uv run ...`	exécuter	`uv export`	vers requirements.txt

2 add pour l'auteur, sync pour les autres

1

uv add

L'auteur ajoute une dépendance : `pyproject.toml`, `uv.lock` et l'environnement sont mis à jour.

2

uv sync

Celui qui récupère le projet installe exactement ce que décrit `uv.lock`, ni plus ni moins.

3

uv run

Vérifie que l'environnement est à jour, le synchronise si besoin, puis exécute.

```
uv run python script.py
uv run pytest
uv run ventes --debut 2025-07
uv run jupyter lab
```

2 La version de Python

```
uv python list              # versions installées ou téléchargeables
uv python install 3.13
uv python pin 3.13          # écrit 3.13 dans .python-version
```

`.python-version`: la version utilisée par le projet
`requires-python`: les versions acceptées ; uv refuse de fixer une version qui ne le respecte pas

Sur les postes, Python 3.12 est déjà installé : `uv init` le détecte et l'inscrit dans `.python-version`, sans rien télécharger.

On ne change de version que si une bibliothèque l'exige.

03

Le fichier pyproject.toml

Sections | Contraintes de version | TP6, étape 1

3 Anatomie d'un pyproject.toml

```
[project]
name = "ventes"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
    "pandas>=2.2",
    "pyarrow>=15",
    "pyyaml>=6",
]

[project.scripts]
ventes = "ventes.cli:main"

[dependency-groups]
dev = ["mypy>=1.10", "pytest>=8", "ruff>=0.6"]

[tool.ruff]
line-length = 120

[build-system]
requires = ["uv_build>=0.12,<0.13"]
build-backend = "uv_build"
```

Section	Contenu
`[project]`	identité, Python, dépendances
`[project.scripts]`	les commandes
`[dependency-groups]`	outils de développement
`[tool.xxx]`	réglages des outils
`[build-system]`	construction du paquet

Les dépendances se modifient avec `uv add` et `uv remove`, pour que le verrou reste cohérent. Les réglages `[tool]` se modifient à la main.

3 Les contraintes de version

Contrainte	Signification
`pandas`	n'importe quelle version
`pandas>=2.2`	2.2 ou plus récente
`pandas>=2.2,<3`	2.x à partir de 2.2
`pandas~=2.2`	idem : compatible 2.2
`pandas==3.0.6`	exactement cette version

Dans `pyproject.toml` : des contraintes **larges**, qui disent avec quoi le code fonctionne. Dans `uv.lock` : les versions **exactes**.

`uv add pandas` écrit la version installée comme minimum, par exemple `pandas>=3.0.6`.

3 Pratique : TP6, étape 1

Créer le projet fil rouge, avec un nom qui dit ce qu'il fait

Terminal

```
uv init --package rapport-ventes
cd rapport-ventes
uv add pandas pyarrow pyyaml
uv add --dev pytest ruff jupyterlab
# recopier les données dans data/brut
uv tree --depth 1
```

La structure de ce matin : `data/brut`, `notebooks`, `src`, `tests`, `outputs`, `config.yaml`
Les données : le dossier `ressources/` du TP6

Résultat attendu de `uv tree --depth 1` : le projet, puis ses dépendances directes ; les outils de développement portent la mention `(group: dev)`.

04

La reproductibilité

uv.lock | Mettre à jour | Développement et production

4 Le fichier uv.lock

La version **exacte** de chaque bibliothèque, dépendances indirectes comprises
L'**empreinte** des fichiers, pour vérifier qu'ils n'ont pas été modifiés
Les variantes pour **chaque système** et chaque version de Python acceptée

On le versionne avec le code. On ne le modifie jamais à la main.

Récupérer un projet

```
git clone
https://github.com/equipe/ventes.git
cd ventes
uv sync
uv run pytest
```

`uv sync --locked` échoue si le verrou n'est plus à jour : idéal dans un pipeline. `uv sync --frozen` installe sans rien vérifier.

4 Mettre à jour, séparer développement et production

Une mise à jour est un choix

```
uv lock --upgrade-package pandas
uv lock --upgrade
uv sync
uv run pytest
```

Développement et production

```
uv add --dev pytest ruff mypy jupyterlab

# sur le serveur
uv sync --no-dev
```

L'environnement de production est plus léger, plus rapide à installer, avec moins de bibliothèques à maintenir.

4 Cohabiter avec pip

Pour quelqu'un qui utilise pip

```
uv export --format requirements-txt \
    --no-dev > requirements.txt
```

Migrer le projet du TP1

```
uv init
uv add -r requirements.txt
```

uv sait aussi exporter au format standard `pylock.toml` : `uv export --format pylock.toml`.

05

Un projet partageable

README | .gitignore | Le test du collègue

5 Un README qui répond aux questions

1 **Quoi**

une phrase

2 **Installer**

`uv sync`

3 **Utiliser**

la commande

4 **Données**

où les déposer

5 **Structure**

les dossiers

6 **Développer**

tests, ruff

```
# ventes

Consolide les ventes mensuelles et calcule
des indicateurs par mois et par région.

## Installation
    uv sync

## Utilisation
    uv run ventes --config config.yaml

## Données
Déposer les fichiers dans data/brut/ventes/,
au format ventes_AAAA_MM.csv.

## Développement
    uv run pytest
```

5 Ce qui se versionne, ce qui ne se versionne pas

Versionné	Non versionné
`pyproject.toml`, `uv.lock`	`.venv/`
`.python-version`	`__pycache__/`
le code, les tests	`outputs/`, régénérable
les notebooks	`.env` et ses secrets
`config.yaml` sans secret	les données volumineuses ou sensibles

.gitignore

```
.venv/
__pycache__/
.ipynb_checkpoints/
.env
outputs/
data/brut/
```

Pour les données non versionnées, le README dit où les trouver et où les déposer.

5 Le test du collègue

1. **Dossier vide** — sans .venv ni outputs
2. **uv sync** — l'environnement
3. **uv run pytest** — les tests
4. **uv run ventes** — le résultat

```
git clone ... /tmp/test-ventes
cd /tmp/test-ventes
uv sync && uv run pytest && uv run ventes
```

Si l'une de ces commandes échoue, le projet n'est pas encore partageable.

TP6 : le projet fil rouge

Cahier des charges dans exercices/tp06-projet-final/README.md

1

Un projet uv

Créé avec `uv init --package` à l'étape 1, dépendances et outils déclarés, `uv.lock` livré.

2

Un notebook d'analyse

Qui importe le paquet, 3 à 5 graphiques commentés et une conclusion.

3

Une commande

Qui produit un rapport HTML, des graphiques ou un fichier Excel, avec journal et code de sortie.

4

Des finitions

Tests, README, `.gitignore`, ruff, et le test du collègue.

Idées : le rapport de la direction commerciale, le suivi des objectifs, l'analyse du catalogue, la fidélité.

TP6 : vérifier le projet

Les chiffres de contrôle

Contrôle	Attendu
fichiers lus, ventes chargées	9 et 2182
ventes hors catalogue	17, produit `P17`
chiffre d'affaires total	409 501,59 euros
lignes mois par région	54
objectif atteint, par mois et région	25 sur 54
région en tête	Île-de-France

Depuis une copie vierge du projet

Commande	Attendu
`uv sync --locked`	sans erreur
`uv run pytest`	tout passe, sans `failed` ni `error`
`uv run ruff check .`	`All checks passed!`
`uv run <commande>`	sorties dans `outputs/`, code 0
`uv run <commande> --config absent.yaml`	message clair, sans trace technique, code 1
le notebook réexécuté	les chiffres de contrôle

Les mêmes données qu'aux TP3 et TP5, donc les mêmes chiffres : à vérifier dans le notebook et dans un test de recette.

La restitution : 5 minutes

1. **Le besoin** — 30 secondes
2. **La démonstration** — 2 minutes
3. **Ce que disent les données** — 1 minute 30
4. **Sous le capot** — 1 minute

Démonstration en direct : lancer la commande devant le groupe et ouvrir le résultat.

Sous le capot : la structure du projet, un choix technique dont on est content, une difficulté rencontrée.

À retenir

- uv remplace venv, pip, pip freeze et la gestion de Python

`uv add` pour l'auteur, `uv sync` pour les autres, `uv run` pour exécuter
`pyproject.toml` : contraintes larges ; `uv.lock` : versions exactes
`uv.lock` se versionne et ne se modifie pas à la main

- Mettre à jour volontairement, puis lancer les tests

Outils dans le groupe `dev`, `uv sync --no-dev` en production
Un README utile, un `.gitignore` complet

- Le test du collègue avant de partager